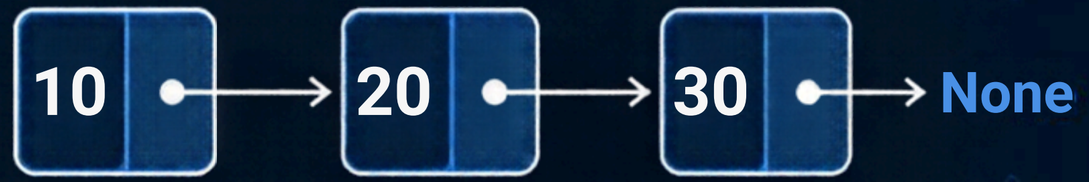
Programación estructurada: listas enlazadas



Autor: Gabriel Omar Carrillo Picado

Profesor: JOSE DURAN GARCIA

Clase: Programación Estructurada

Este fue el primer paso

Creé la función de las listas enlazadas y aprendí que cada una es un nodo independiente.

Cada llamada a la función crea un nodo que guarda un dato y deja el enlace al siguiente en None.

Entendí que un nodo es una unidad independiente de la lista.
La función crea el nodo, guarda el dato y deja preparado el enlace hacia el siguiente nodo.

4. Crear el nodo

Crea el archivo lista_enlazada.py. Agrega esta función. Cada llamada produce un nodo independiente.

Copiado

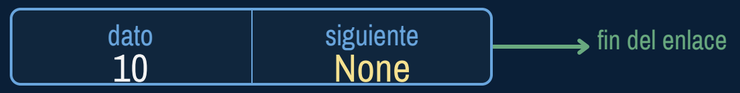
```
def crear_nodo(dato):
  return {"dato": dato, "siguiente": None}
```

Pausa de comprensión: dibuja los enlaces antes y después de esta operación e identifica cuándo termina el recorrido.

☑ Completé y comprendí esta etapa

DIAGRAMA TÉCNICO · NODO INDEPENDIENTE

crear_nodo(10) crea una unidad independiente: guarda el dato y prepara el siguiente enlace.

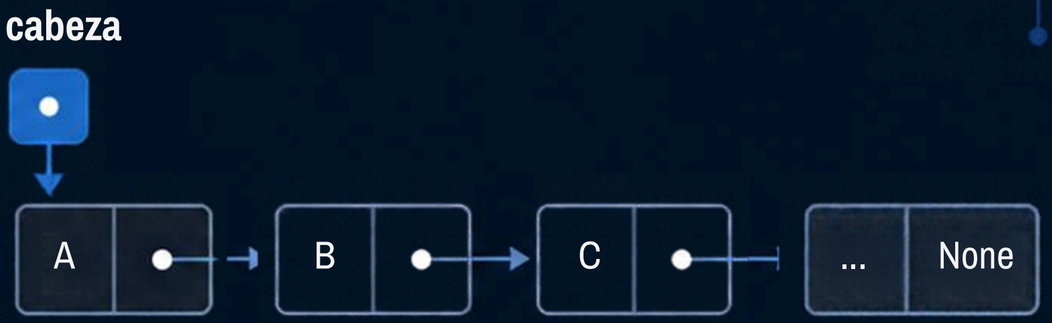


Paso 2: Insertar al inicio

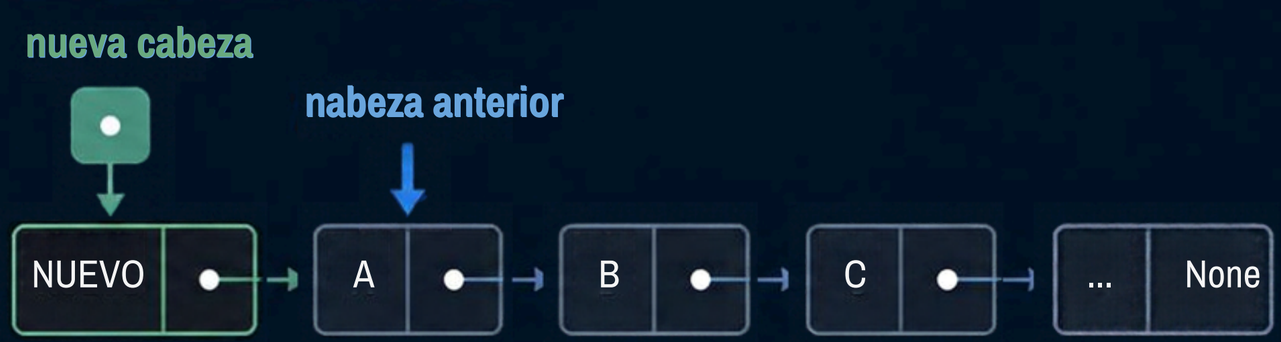
La nueva función se conecta con la cabeza anterior y devuelve la nueva cabeza.

```
def insertar_inicio(cabeza, dato):
    nuevo = crear_nodo(dato)
    nuevo["siguiente"] = cabeza
    return nuevo
```

ANTES DE LA INSERCIÓN



DESPUÉS DE INSERTAR AL INICIO



REFLEXIÓN

Entendí que este orden evita perder el acceso al resto de la lista, porque el nuevo nodo primero apunta a la cabeza anterior y luego se convierte en la nueva cabeza.

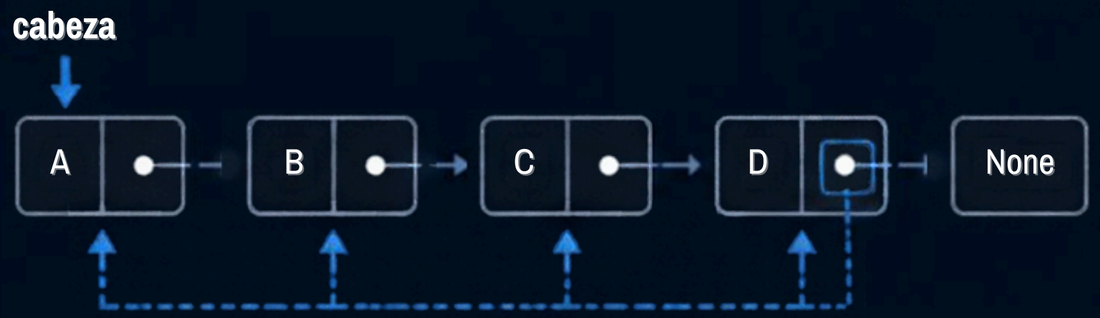
Paso 3: Insertar al final

Si la lista está vacía, el nuevo nodo pasa a ser la cabeza. Si tiene elementos, se avanza hasta el último nodo y se conecta con el nuevo.

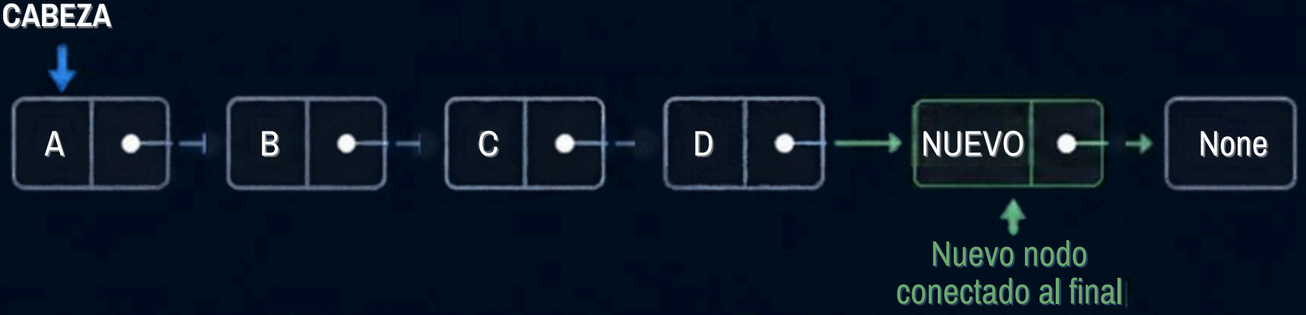
```python
def insertar_final(cabeza, dato):
    nuevo = crear_nodo(dato)
    if cabeza is None:
        return nuevo

    actual = cabeza
    while actual["siguiente"] is not None:
        actual = actual["siguiente"]
    actual["siguiente"] = nuevo
    return cabeza
```

RECORRIDO HASTA EL ÚLTIMO NODO



CONECTAR EL NUEVO NODO



REFLEXIÓN

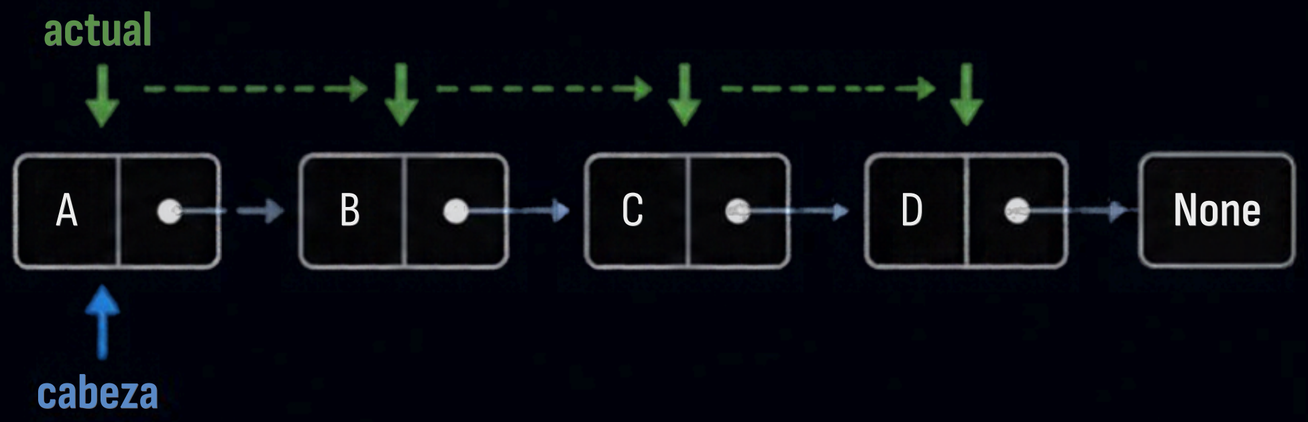
Entendí que primero se debe comprobar si la lista está vacía. Si no lo está, hay que recorrerla hasta encontrar el último nodo para enlazar allí el nuevo nodo.

Paso 4: Recorrer y mostrar

La nueva función utiliza la variable actual, que avanza por los nodos sin cambiar la cabeza de la lista. Si la lista está vacía, se imprime None.

```
def mostrar(cabeza):
    actual = cabeza
    while actual is not None:
    print(actual["dato"], end=" -> ")
    actual = actual["siguiente"]
print("None")
```

RECORRIDO CON LA VARIABLE ACTUAL



- **actual** avanza de nodo en nodo hasta llegar a **None**.
- **cabeza** permanece en el primer nodo; no cambia.

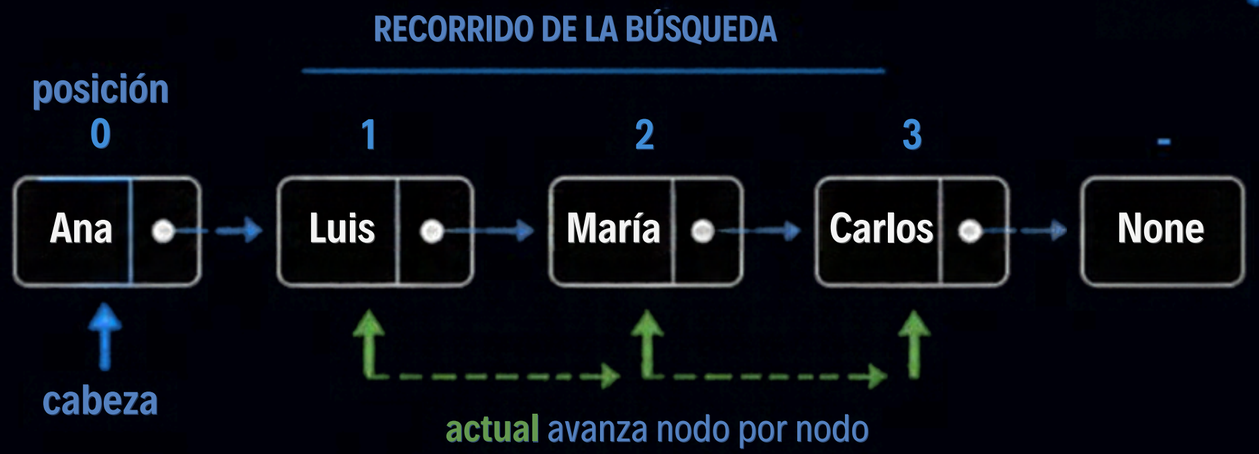
REFLEXIÓN

Entendí que la variable actual sirve para avanzar por cada nodo sin modificar la cabeza. De esta manera puedo recorrer y mostrar la lista, y cuando está vacía el resultado es None.

Paso 5: Buscar un turno

La función buscar recorre la lista desde la posición 0 y devuelve la primera posición encontrada. Si el nombre no existe, devuelve −1.

```
def buscar(cabeza , dato):
 actual = cabeza
 posición = 0
 while actual is not None:
  if actual[“dato“] == dato:
     return posicion
  actual = actual[“siguiente“]
  posición += 1
 return -1
```



COINCIDENCIA ENCONTRADA

Si actual["dato"] == dato, se devuelve la posición actual.
Ejemplo: buscar("María") devuelve 2.

SIN COINCIDENCIA

Si se recorre toda la lista y no se encuentra el nombre, se devuelve -1.
Ejemplo: buscar("Pedro") devuelve -1.

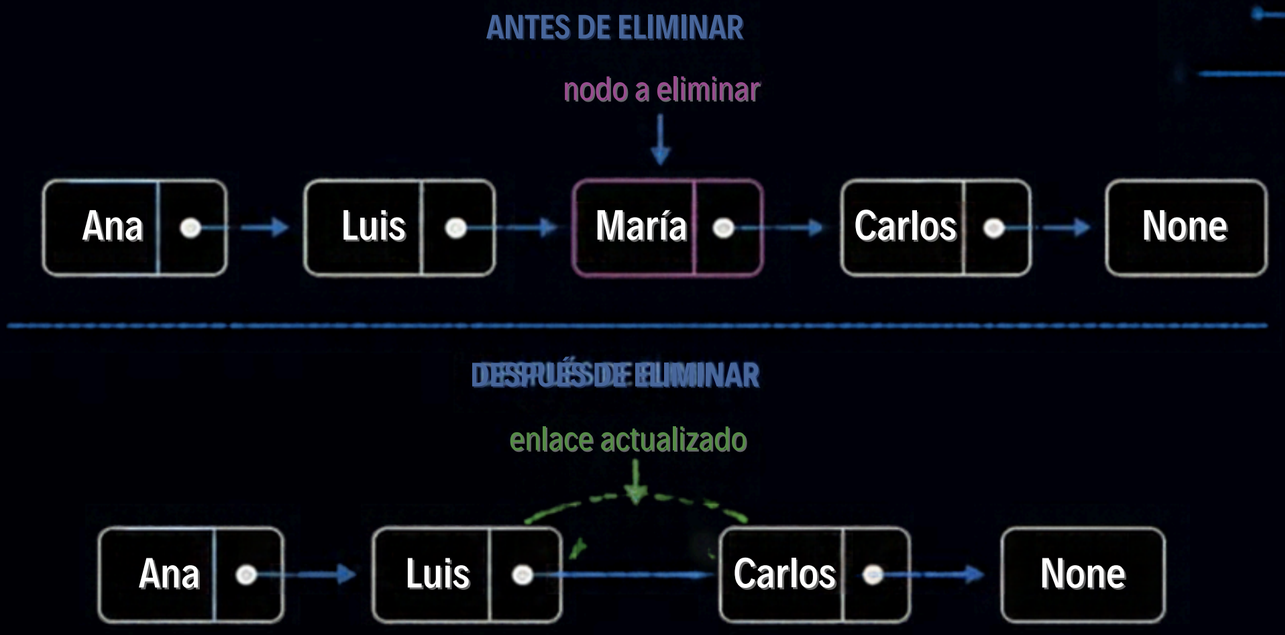
REFLEXIÓN

Entendí que la búsqueda empieza desde la posición 0 y avanza nodo por nodo.
Si encuentra el nombre, devuelve la primera posición en la que aparece;
si recorre toda la lista y no lo encuentra, devuelve -1.

Paso 6: Eliminar un nodo

La función distingue si la lista está vacía, si se elimina la cabeza o si se elimina otro nodo. Devuelve una tupla con la nueva cabeza y una confirmación.

```python
def eliminar(cabeza, dato):
  if cabeza is None:
    return cabeza, False
  if cabeza["dato"] = dato:
    return cabeza["siguiente"], True
  anterior = cabeza
  actual = cabeza["siguiente"]
  while actual is not None:
    if actual["dato"] = dato:
      anterior["siguiente"] = actual["siguiente"]
      return cabeza, True
      anterior = actual
      actual = actual["siguiente"]
      return cabeza, False
```



LISTA VACÍA

Si la lista está vacía (cabeza is None, no se elimina nada y se devuelve (cabeza, False).

ELIMINAR LA CABEZA

Si el dato a eliminar está en la cabeza, la nueva cabeza será el nodo siguiente. Se devuelve (nueva cabeza, True).

ELIMINAR OTRO NODO

Si el dato está en otro nodo, se reemplaza el enlace del nodo anterior por el siguiente del nodo eliminado. Se devuelve (cabeza, True).

REFLEXIÓN

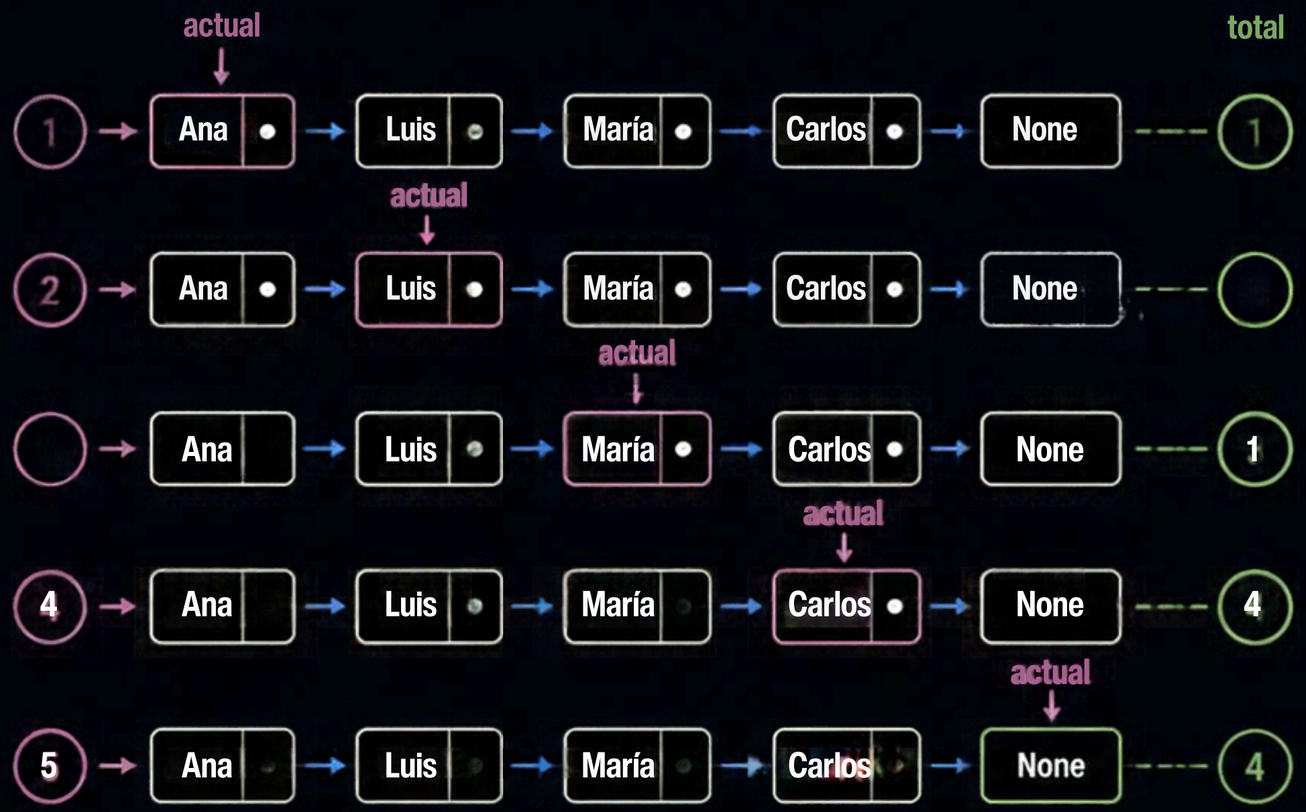
Entendí por para eliminar un nodo debo revisar primero si la lista está vacía. Si elimino la cabeza, la nueva cabeza será el nodo siguiente. Si elimino otro nodo, reenlazo el enlace del nodo anterior por el siguiente del nodo eliminado, y así el nodo queda excluido de la lista. También entendí que la función devuelve la nueva cabeza y una confirmación para indicar si la eliminación se realizó.

Paso 7: Contar los nodos

La función cuenta los nodos recorriendo la estructura completa.
No utiliza un contador global que pueda quedar desactualizado.

```python
def contar(cabeza):
    total = 0
    actual = cabeza
    while actual is not None:
        total += 1
        actual = actual["siguiente"]
    return total
```

RECORRIDO DE LA LISTA Y CONTADOR



El recorrido termina cuando actual es None y se devuelve total = 4.

REFLEXIÓN

Entendí que la cantidad de nodos se obtiene recorriendo la lista desde la cabeza hasta None y aumentando el contador en cada nodo. Así el resultado se calcula con el estado real de la estructura y no depende de un contador global que podría quedar desactualizado.

0004

Paso 8: Integrar el menú final

EJECUCIÓN LOCAL
o en Colab

PROGRAMA COMPLETO · MENÚ PRINCIPAL

```
def menu():
    cabeza = None
    while True:
        print("\n--- MENÚ LISTA ENLAZADA ---")
        print("1. Insertar al inicio")
        print("2. Insertar al final")
        print("3. Buscar   4. Eliminar")
        print("5. Mostrar  6. Contar  0. Salir")
        op = input("Seleccione una opción: ")
        if op == "1":
            cabeza = insertar_inicio(cabeza, input("Dato: "))
        elif op == "2":
            cabeza = insertar_final(cabeza, input("Dato: "))
        elif op == "3": print(buscar(cabeza, input("Dato: ")))
        elif op == "4":
            cabeza, ok = eliminar(cabeza, input("Dato: "))
            print("¿Se eliminó?", ok)
        elif op == "5": mostrar(cabeza)
        elif op == "6": print("Total:", contar(cabeza))
        elif op == "0": break
        else: print("Opción no válida")
    return cabeza

if __name__ == "__main__": cabeza = menu()
```

PASOS 4–10 + ESTE BLOQUE

Las funciones de los pasos 4 al 10 + este bloque de menú forman el programa completo y listo.

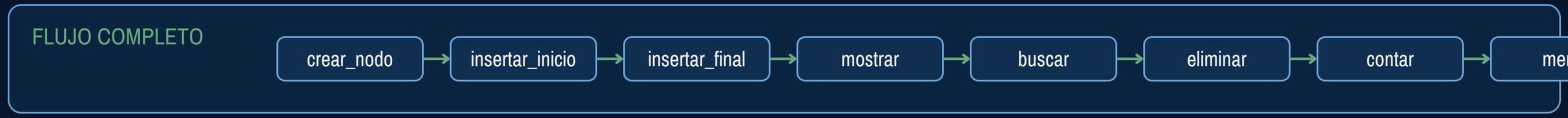
El menú conecta insertar, buscar, eliminar, mostrar y contar.

La variable cabeza conserva el retorno de cada función para actualizar la lista correctamente.

Código final ejecutable: sin instrucciones temporales ni salida de prueba.

CIERRE · DIAGRAMA DE CONTROL DE LA LISTA

Una lista enlazada funciona cuando cada retorno conserva el acceso a la siguiente decisión.



ERRORES FRECUENTES

No actualizar cabeza al insertar o eliminar.

Avanzar sin comprobar actual is not None.

Confundir dato con siguiente.

EVIDENCIAS DE APRENDIZAJE

Cada función devuelve un estado claro.

Los diagramas muestran cómo cambian los enlaces.

El menú integra las operaciones.

PREGUNTAS DE REFLEXIÓN

¿Qué ocurre si la lista está vacía?

¿Por qué eliminar la cabeza devuelve otra cabeza?

¿Cómo se conserva el acceso a la lista?

10 ⟶ 20 ⟶ 30 ⟶ None

Del nodo al menú: comprender, recorrer, modificar y verificar.